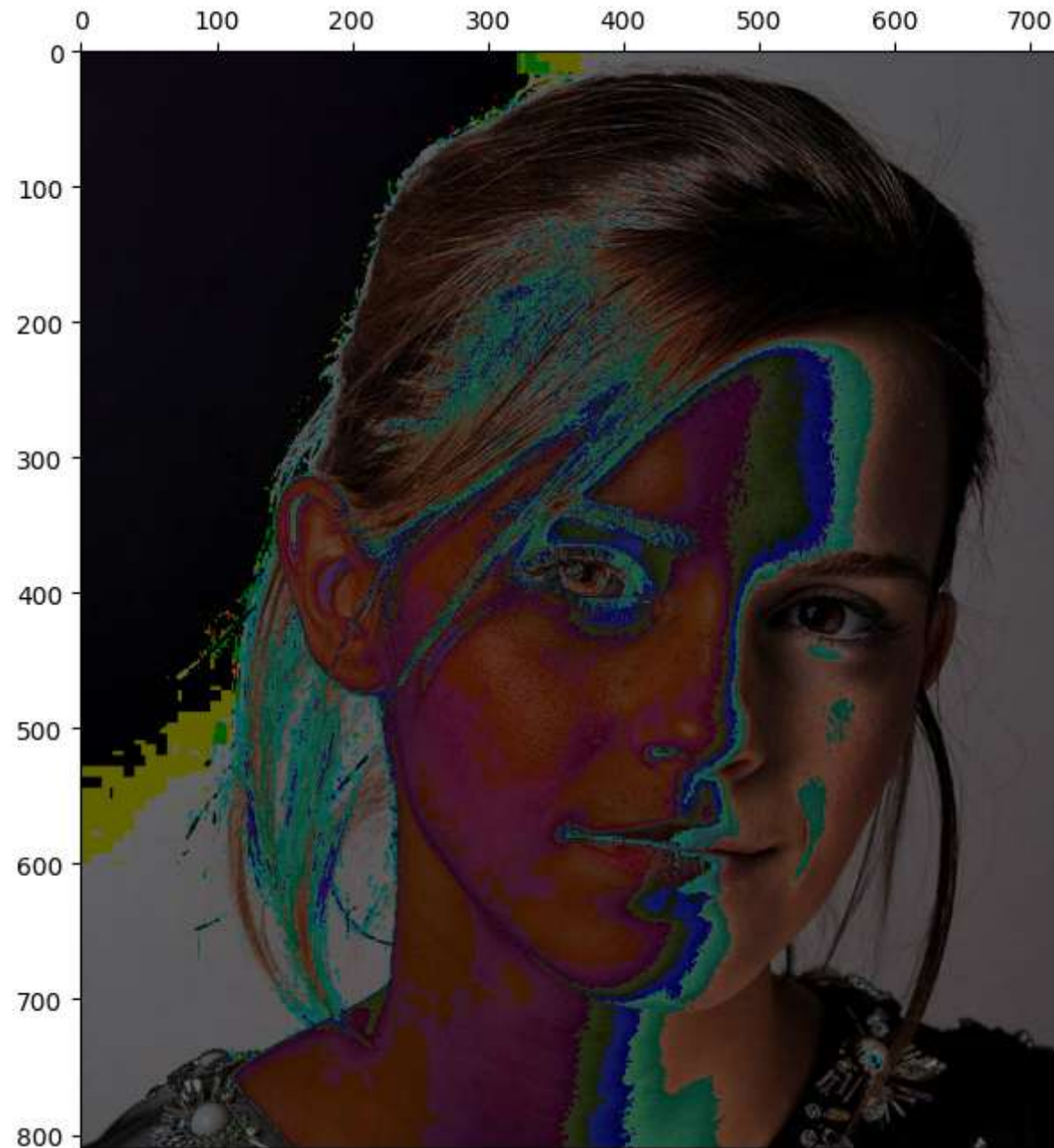


```
In [1]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve2d
```

```
In [2]: f = cv.imread("images/emma.jpg")
# cv.namedWindow('Image', cv.WINDOW_AUTOSIZE)
# cv.imshow('Image', f)
# cv.waitKey(0)
# cv.destroyAllWindows()
```

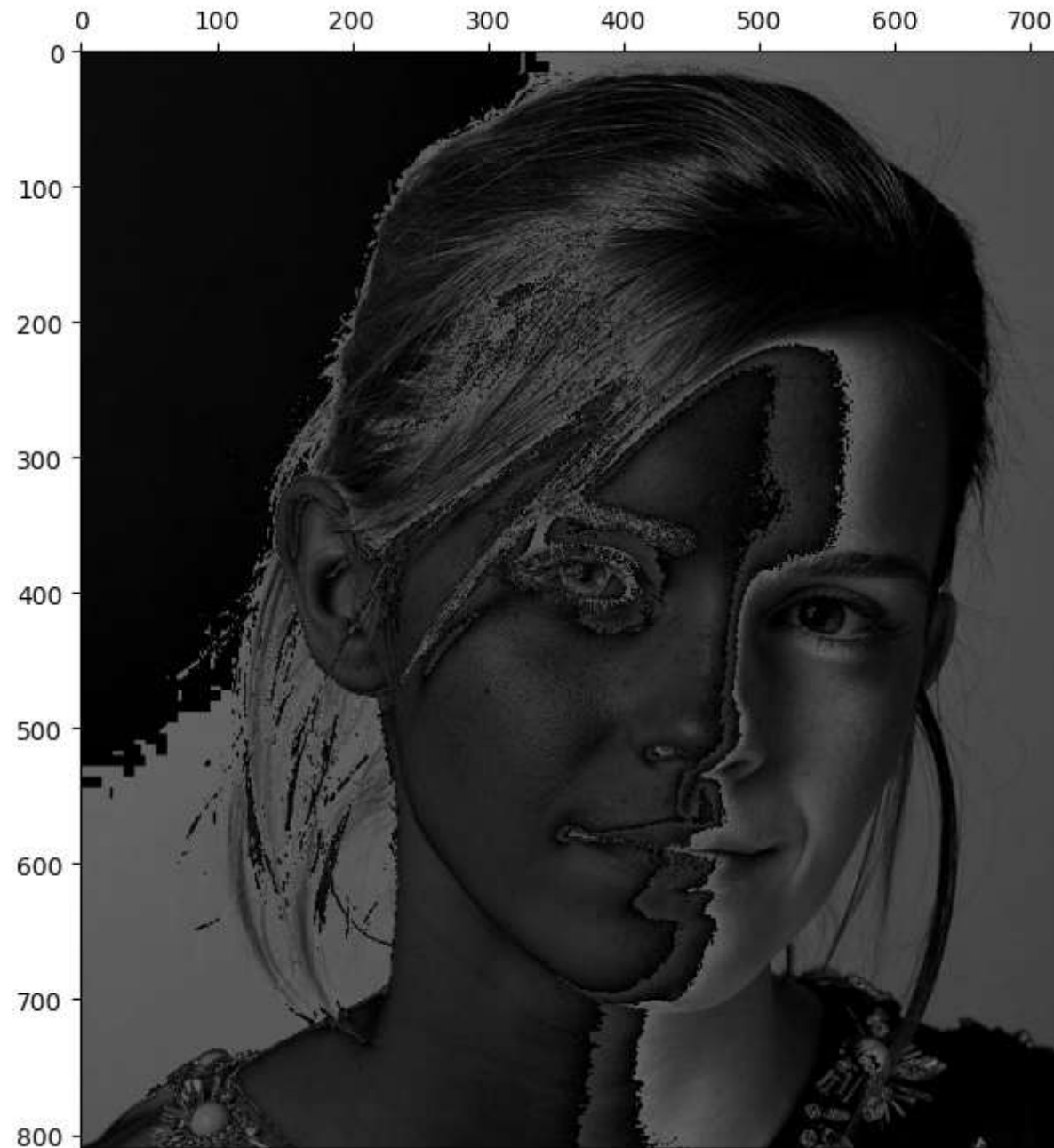
```
In [3]: fig, ax = plt.subplots(1,1,figsize=(8,8))
t1 = np.arange(100, dtype=np.uint8)
t2 = np.arange(50, dtype=np.uint8)
t3 = np.arange(106, dtype=np.uint8)
t = np.concatenate((t1,t2,t3))
print(t.shape)
f = t[f]
ax.imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
ax.xaxis.set_ticks_position('top')
plt.show()
```

(256,)



```
In [4]: f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)
fig, ax = plt.subplots(1,1,figsize=(8,8))
t1 = np.arange(100, dtype=np.uint8)
t2 = np.arange(50, dtype=np.uint8)
t3 = np.arange(106, dtype=np.uint8)
t = np.concatenate((t1,t2,t3))
print(t.shape)
f = t[f]
ax.imshow(f, cmap='gray', vmin=0, vmax=255)
ax.xaxis.set_ticks_position('top')
plt.show()
```

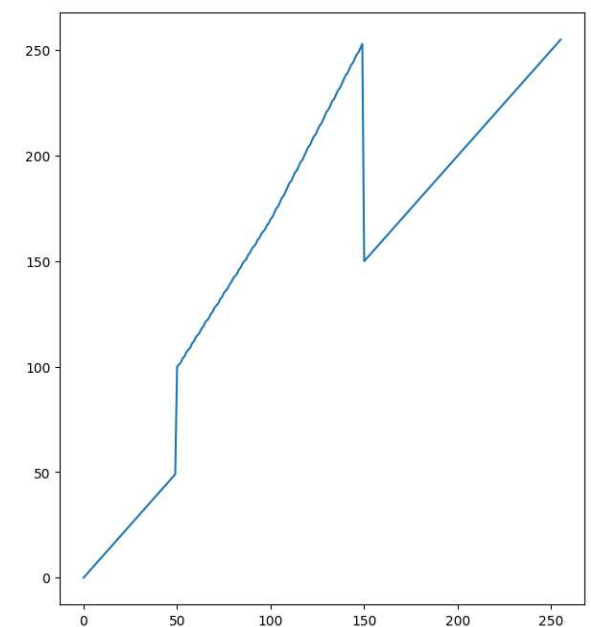
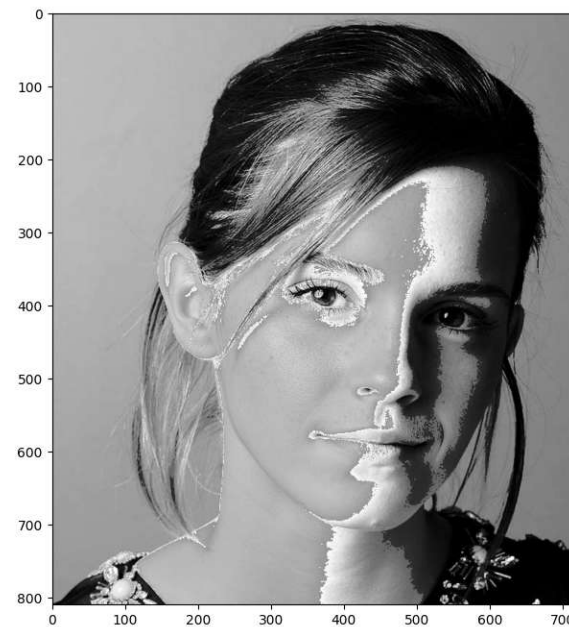
(256,)



```
In [5]: def intensity_transform(im, breakpoints):  
    # 1. Unpack your breakpoints into two separate arrays: x_coords (input) and y_coords (output).  
  
    # 2. Use np.interp() to evaluate the original image pixels against your coordinates.  
    # Concept: np.interp(im, x_coords, y_coords)  
  
    # 3. The interpolation returns floats. Convert the array back to standard image format using .astype(np.uint8).  
  
    # return the final_image  
    return
```

```
In [6]: x = [0,50,50,100,150,150,255]  
y = [0,50,100,170,255,150,255]  
f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)  
X = np.arange(256, dtype='uint8')  
F = np.interp(X,x,y).astype(np.uint8)  
  
print(F)  
fig, axes = plt.subplots(1,3, figsize=(24,8))  
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)  
axes[1].imshow(F[f], cmap='gray', vmin=0, vmax=255)  
axes[2].plot(X,F)  
plt.show()  
  
#print(X_interp)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
 36 37 38 39 40 41 42 43 44 45 46 47 48 49 100 101 102 104
105 107 108 109 111 112 114 115 116 118 119 121 122 123 125 126 128 129
130 132 133 135 136 137 139 140 142 143 144 146 147 149 150 151 153 154
156 157 158 160 161 163 164 165 167 168 170 171 173 175 176 178 180 181
183 185 187 188 190 192 193 195 197 198 200 202 204 205 207 209 210 212
214 215 217 219 221 222 224 226 227 229 231 232 234 236 238 239 241 243
244 246 248 249 251 253 150 151 152 153 154 155 156 157 158 159 160 161
162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179
180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197
198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215
216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233
234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251
252 253 254 255]
```



```
In [7]: def intensity_transform(im, breakpoints):
        x, y = zip(*breakpoints)
        X = np.arange(256, dtype='uint8')
        F = np.interp(X, x, y).astype(np.uint8)
        return F[im], X, F
```

```
In [8]: f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)

# 3. Experiment with breakpoints (S-curve for pleasing contrast enhancement)
breakpoints = [(0, 0), (64, 32), (128, 128), (192, 224), (255, 255)]

# 4. Apply the transformation
transformed_img, X, F = intensity_transform(f, breakpoints)

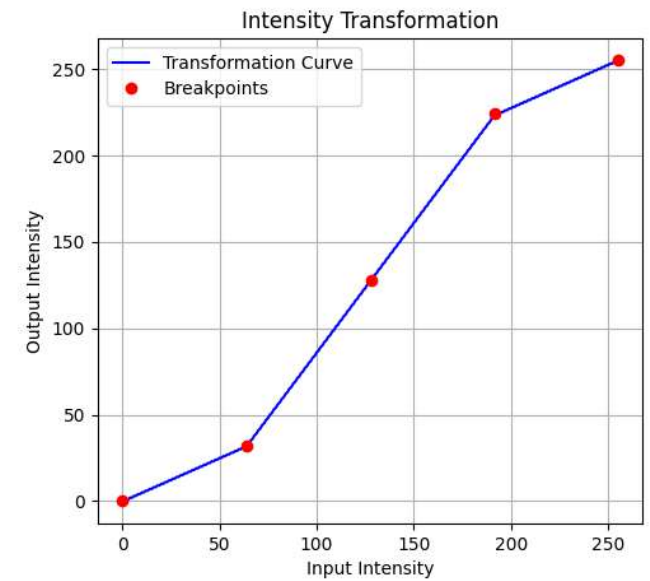
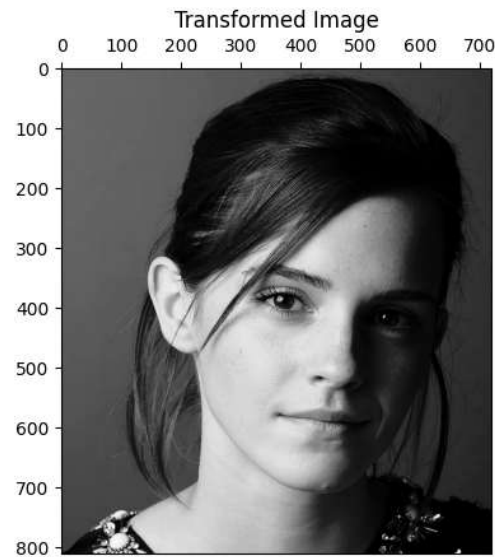
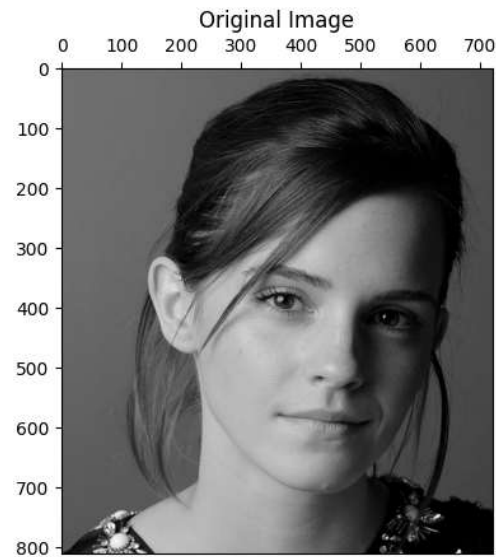
# 5. Display the original image, transformed image, and plot side-by-side
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
# Move x-axis ticks to the top for all subplots

# Original Image
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].xaxis.set_ticks_position('top') # Hides the axes ticks for a cleaner look

# Transformed Image
axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Transformed Image")
axes[1].xaxis.set_ticks_position('top') # Hides the axes ticks for a cleaner look

# Transformation Plot
axes[2].plot(X, F, color='blue', label='Transformation Curve')
# Optionally plot the actual breakpoints as red dots for clarity
x_coords, y_coords = zip(*breakpoints)
axes[2].plot(x_coords, y_coords, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()

plt.tight_layout()
plt.show()
```

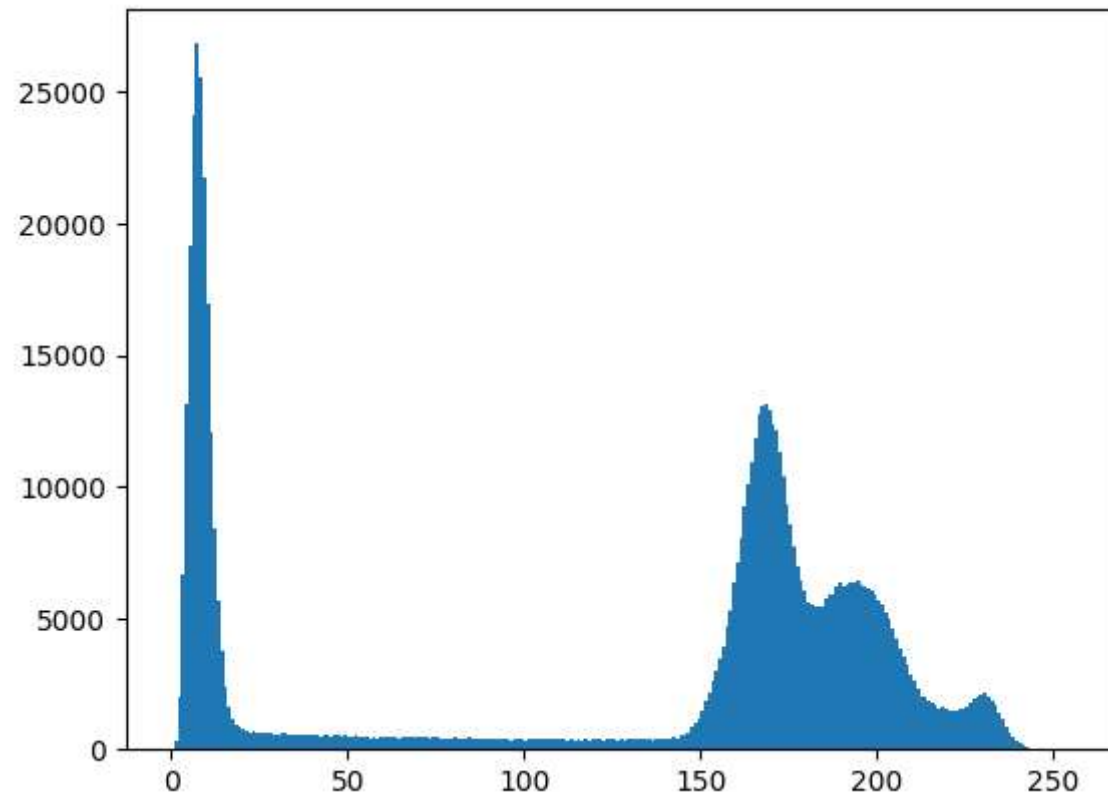


```
In [9]: brain_img = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)

# Compute histogram
hist_values, bin_edges, _ = plt.hist(brain_img.ravel(), bins=256, range=[0,256])

plt.show()

# Print histogram values
for intensity in range(140,256):
    print(f"Intensity {intensity}: {100*hist_values[intensity]/np.max(hist_values)}")
```



Intensity 140: 1.5274000670565884
Intensity 141: 1.5124986029877436
Intensity 142: 1.8403308125023283
Intensity 143: 1.6130834854524456
Intensity 144: 1.9595425250530865
Intensity 145: 2.1122825317587455
Intensity 146: 2.4363893752561188
Intensity 147: 3.1069552583541333
Intensity 148: 3.7886972395037812
Intensity 149: 4.444361658532951
Intensity 150: 5.558246097679097
Intensity 151: 7.0111388443914615
Intensity 152: 8.046790597176173
Intensity 153: 9.659874082628619
Intensity 154: 11.187274149685207
Intensity 155: 12.755653242931118
Intensity 156: 14.644413813657192
Intensity 157: 17.434712960548374
Intensity 158: 19.666207204857876
Intensity 159: 23.782736653876242
Intensity 160: 26.632641657042804
Intensity 161: 29.92959058227471
Intensity 162: 34.418656633014194
Intensity 163: 37.43992847297247
Intensity 164: 40.56923592742987
Intensity 165: 44.08970681369445
Intensity 166: 47.509592817494315
Intensity 167: 48.7687665313117
Intensity 168: 49.036992884550905
Intensity 169: 48.21741236076444
Intensity 170: 46.19826397943598
Intensity 171: 45.18496442275453
Intensity 172: 42.17114331483068
Intensity 173: 38.61341876839399
Intensity 174: 34.61237566590918
Intensity 175: 31.825801885035204
Intensity 176: 28.826882241180197
Intensity 177: 25.973251871996425
Intensity 178: 23.808814215996723
Intensity 179: 22.437879521663003
Intensity 180: 20.873225794434305

Intensity 181: 20.411280408300115
Intensity 182: 20.23246283947398
Intensity 183: 20.36285065007637
Intensity 184: 20.381477480162427
Intensity 185: 21.39477703684387
Intensity 186: 21.912602913236224
Intensity 187: 22.095145848079575
Intensity 188: 23.08236784264054
Intensity 189: 23.689602503445965
Intensity 190: 23.071191744588905
Intensity 191: 23.324516633759266
Intensity 192: 23.730581529635288
Intensity 193: 23.61509518310174
Intensity 194: 23.946652758633537
Intensity 195: 23.488432738516558
Intensity 196: 23.15314979696755
Intensity 197: 22.721007338971052
Intensity 198: 22.430428789628582
Intensity 199: 21.96103267145997
Intensity 200: 21.126550683604663
Intensity 201: 20.601274075177887
Intensity 202: 19.42033304772194
Intensity 203: 18.492716909436353
Intensity 204: 17.20374026748128
Intensity 205: 15.683790932459114
Intensity 206: 14.17129232947137
Intensity 207: 13.239950825168572
Intensity 208: 12.07763662779868
Intensity 209: 10.647096077189584
Intensity 210: 9.712029206869575
Intensity 211: 8.657750623998808
Intensity 212: 7.424654472301904
Intensity 213: 7.525239354766605
Intensity 214: 6.82114517751369
Intensity 215: 6.593897850463808
Intensity 216: 6.415080281637671
Intensity 217: 5.8860783071936815
Intensity 218: 6.016466117796074
Intensity 219: 5.796669522780613
Intensity 220: 5.50609097343814
Intensity 221: 5.502365607420929

Intensity 222: 5.5768729277651525
Intensity 223: 5.822747084901091
Intensity 224: 5.815296352866669
Intensity 225: 6.053719777968185
Intensity 226: 6.698208098945721
Intensity 227: 7.067019334649629
Intensity 228: 7.447006668405171
Intensity 229: 7.648176433334575
Intensity 230: 7.97600864284916
Intensity 231: 7.6034720411280405
Intensity 232: 6.962709086167716
Intensity 233: 6.187832954587789
Intensity 234: 5.226688522147301
Intensity 235: 4.224565063517491
Intensity 236: 3.2820474611630592
Intensity 237: 2.3097269306709385
Intensity 238: 1.8924859367432851
Intensity 239: 1.2740751778862274
Intensity 240: 0.8307566218380956
Intensity 241: 0.5364527064784115
Intensity 242: 0.30548001341131764
Intensity 243: 0.15646537272287003
Intensity 244: 0.10058488246470215
Intensity 245: 0.06705658830980143
Intensity 246: 0.022352196103267146
Intensity 247: 0.022352196103267146
Intensity 248: 0.0
Intensity 249: 0.0
Intensity 250: 0.0
Intensity 251: 0.0
Intensity 252: 0.0
Intensity 253: 0.0
Intensity 254: 0.0
Intensity 255: 0.0

```
In [10]: f = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)
print(f)
breakpoints = [(0, 0), (153, 0), (168, 255), (185, 0), (221, 0)]

transformed_img, X, F = intensity_transform(f, breakpoints)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

```

axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].xaxis.set_ticks_position('top')

axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Accentuate White Matter")
axes[1].xaxis.set_ticks_position('top')

axes[2].plot(X, F, color='blue', label='Transformation Curve')
x_coords, y_coords = zip(*breakpoints)
axes[2].plot(x_coords, y_coords, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()

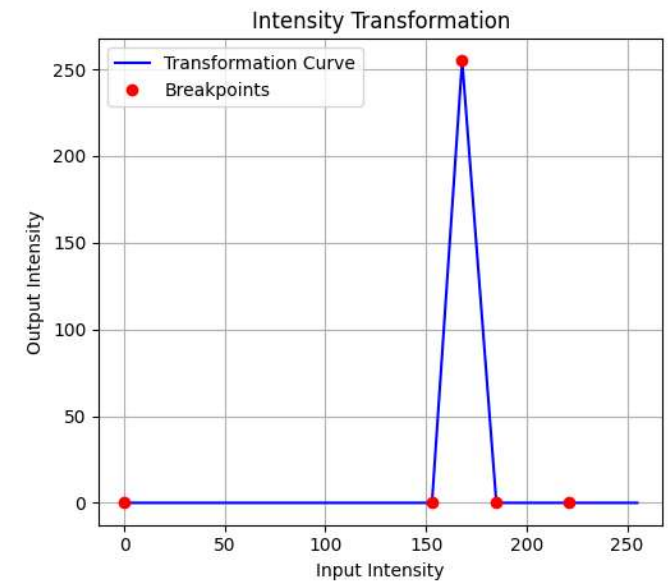
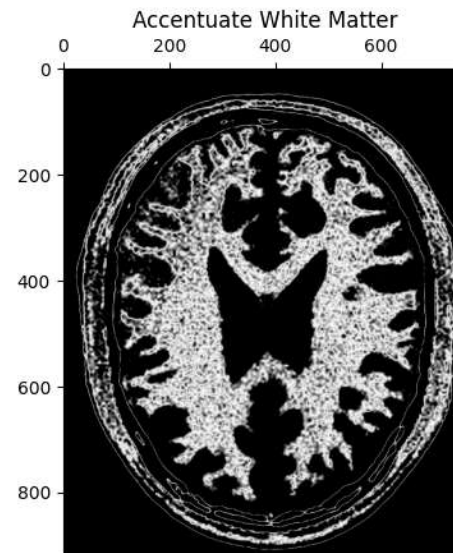
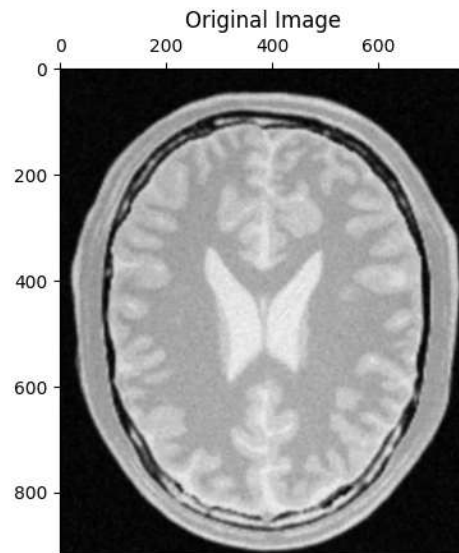
plt.tight_layout()
plt.show()

```

```

[[ 2  4  6 ... 10 12 11]
 [ 4  5  7 ... 10 11 11]
 [ 6  7  8 ...  9 10 10]
 ...
 [ 8  9  9 ...  4  4  4]
 [ 8  9  9 ...  4  4  4]
 [ 9  9  9 ...  4  4  4]]

```



```
In [11]: f = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)
print(f)
breakpoints = [(0, 0), (185, 0), (194, 255), (222, 0), (255, 0)]

transformed_img, X, F = intensity_transform(f, breakpoints)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].xaxis.set_ticks_position('top')

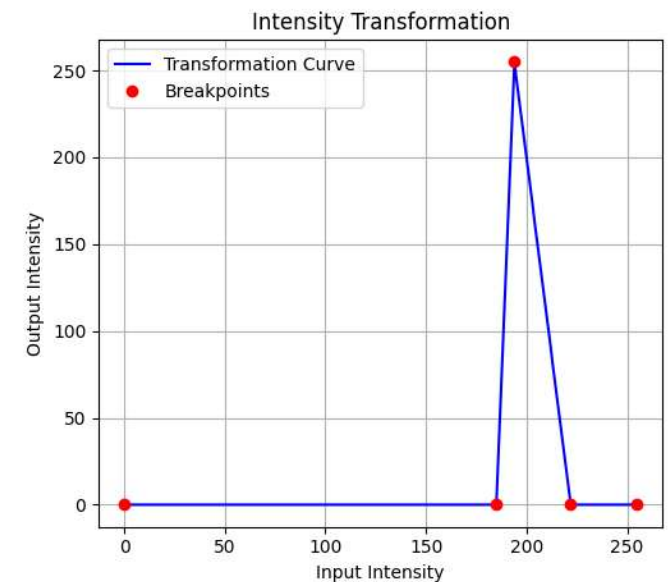
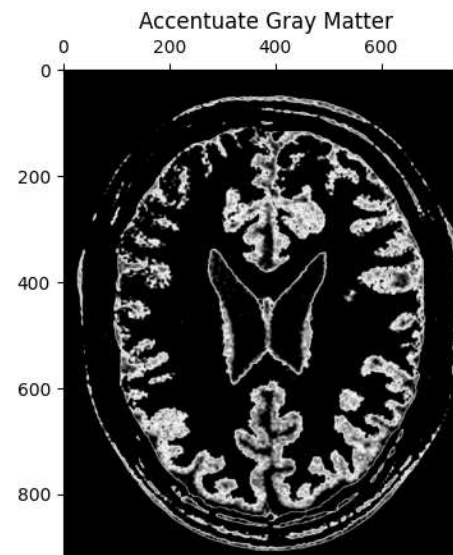
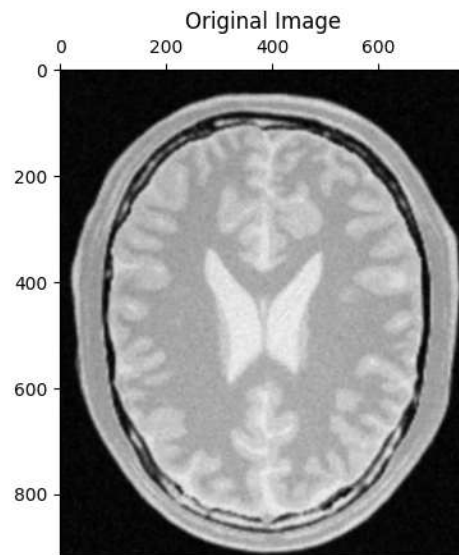
axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Accentuate Gray Matter")
axes[1].xaxis.set_ticks_position('top')

axes[2].plot(X, F, color='blue', label='Transformation Curve')
x_coors, y_coors = zip(*breakpoints)
axes[2].plot(x_coors, y_coors, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
```

```
axes[2].grid(True)
axes[2].legend()
```

```
plt.tight_layout()
plt.show()
```

```
[[ 2  4  6 ... 10 12 11]
 [ 4  5  7 ... 10 11 11]
 [ 6  7  8 ...  9 10 10]
 ...
 [ 8  9  9 ...  4  4  4]
 [ 8  9  9 ...  4  4  4]
 [ 9  9  9 ...  4  4  4]]
```



Interpretation of Question 2: Tissue Accentuation

To successfully accentuate specific tissues in the brain proton density image, I first plotted a flattened histogram to identify the distinct intensity clusters.

In a proton density (PD) MRI slice, image intensity is dictated by water concentration. Therefore, the physical appearance of the tissues is inverted compared to their anatomical names:

- **White Matter:** Composed largely of fatty myelin, it has lower water content and yields a weaker signal. It appears darker on the PD scan. On the histogram, I identified this as the first major peak. I designed a triangular breakpoint filter ranging from 153 to 185, peaking at an intensity of **168**.
- **Gray Matter:** Contains more cell bodies and higher water content, yielding a stronger signal and appearing brighter. I identified this as the second peak on the histogram. I designed a triangular breakpoint filter ranging from 185 to 222, peaking at an intensity of **194**.

By applying these custom breakpoint coordinates to the `intensity_transform` function, the target matter is driven to 255 (white) while all other background and tissue values are suppressed to 0 (black).

```
In [12]: img_original = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\white.png", cv.IMREAD_COLOR)
f = cv.cvtColor(img_original, cv.COLOR_BGR2LAB)
L,a,b = cv.split(f)
gamma = 0.5
Lc = (((L/255.0)**(gamma))*255).astype(np.uint8)
img_lab = cv.merge([Lc,a,b])
img = cv.cvtColor(img_lab, cv.COLOR_LAB2RGB)

# Create a figure with 2 subplots side-by-side
# Create a figure with a 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(12, 9))

# 1. Top-Left: Original Image
axes[0, 0].imshow(cv.cvtColor(img_original, cv.COLOR_BGR2RGB))
axes[0, 0].set_title("Original Image")
axes[0, 0].axis('off') # Hides the axes and border completely

# 2. Top-Right: Edited Image
axes[0, 1].imshow(img)
axes[0, 1].set_title(f"Gamma Corrected Image (Gamma = {gamma})")
axes[0, 1].axis('off')

# 3. Bottom-Left: Histogram for the Original L plane
axes[1, 0].hist(L.ravel(), bins=256, range=[0, 256], color='gray')
axes[1, 0].set_title("Histogram: Original L Plane")
axes[1, 0].set_xlabel("Pixel Intensity")
axes[1, 0].set_ylabel("Frequency")

# 4. Bottom-Right: Histogram for the Corrected L plane
```

```
axes[1, 1].hist(Lc.ravel(), bins=256, range=[0, 256], color='gray')
axes[1, 1].set_title(f"Histogram: Corrected L Plane (Gamma = {gamma})")
axes[1, 1].set_xlabel("Pixel Intensity")
axes[1, 1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()
```

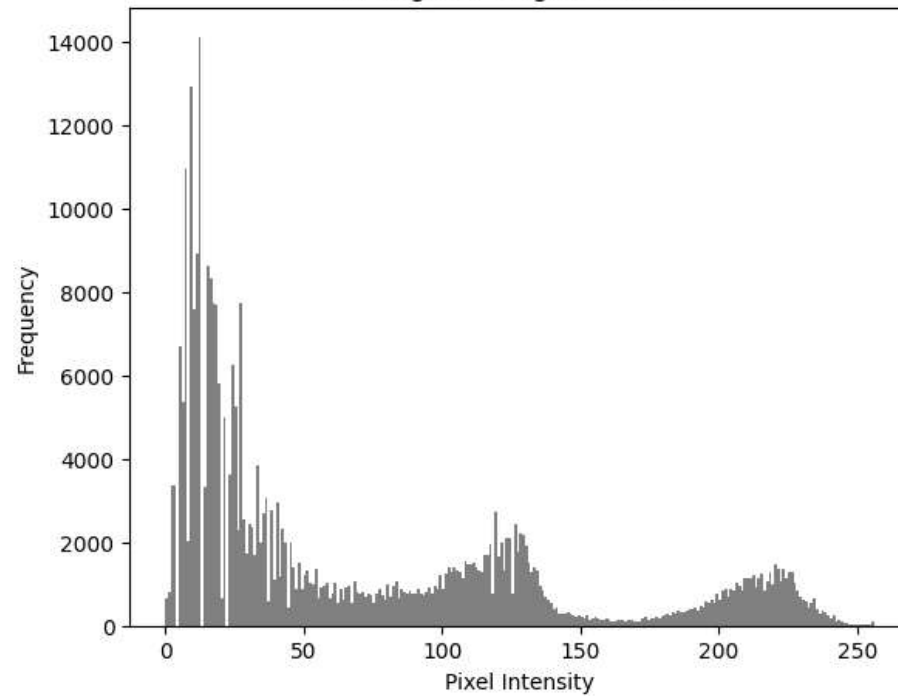
Original Image



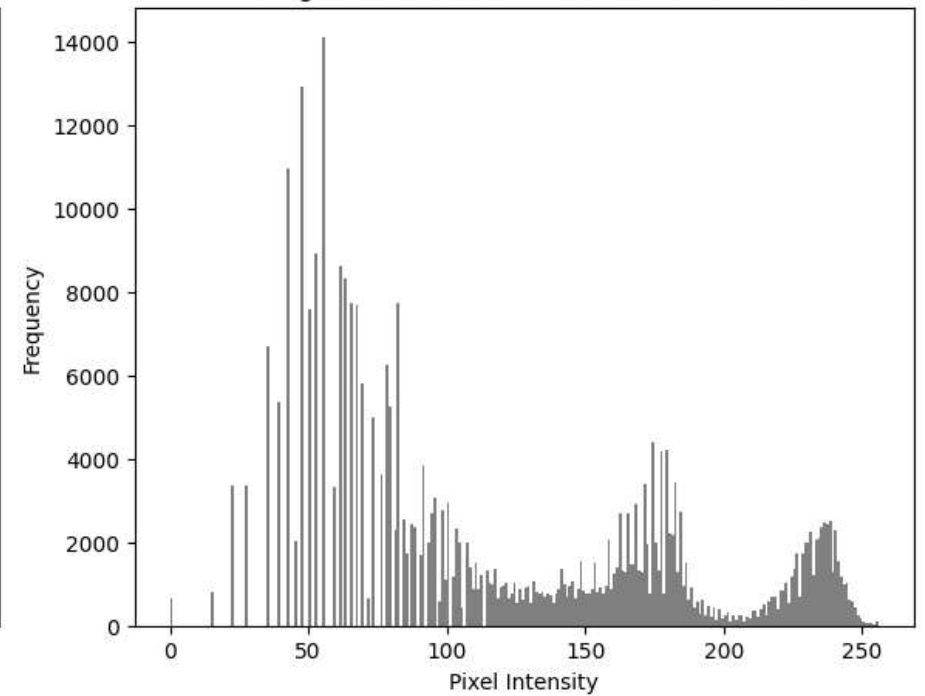
Gamma Corrected Image (Gamma = 0.5)



Histogram: Original L Plane



Histogram: Corrected L Plane (Gamma = 0.5)

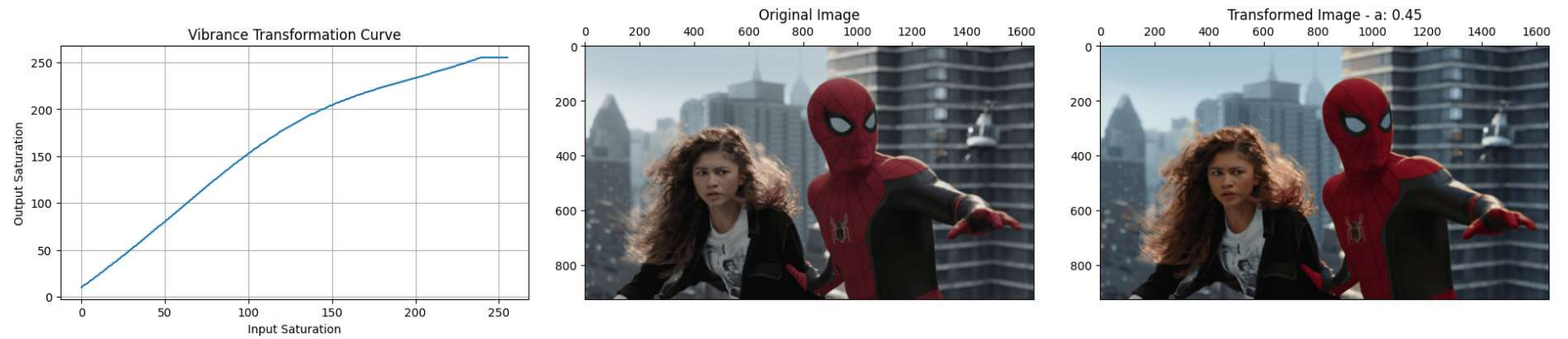


Interpretation of Question 3: Gamma Correction

The goal of this operation was to reveal the shadow details in the underexposed rocks without distorting the colors of the woman's skin or dress. To achieve this, the image was converted to the $L^*a^*b^*$ color space, separating the luminance (L^*) from the chrominance (a^* , b^*).

A gamma correction of $\gamma = 0.5$ was applied strictly to the normalized L plane. Because $\gamma < 1$, this non-linear power-law transformation stretches the lower intensity values. As shown in the histograms, the heavy spike of dark pixels near 0 in the original L plane is spread out towards the mid-tones in the corrected L plane, effectively brightening the shadows while maintaining the original color data.

```
In [13]: f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\spider.png", cv.IMREAD_COLOR)
img = cv.cvtColor(f, cv.COLOR_BGR2HSV)
h,s, v = cv.split(img)
sigma = 70
a = 0.45
i = np.arange(256)
F = i + a * (128 * np.exp(-((i-128)**2) / (2*sigma**2)))
F = np.clip(F,0,255).astype(np.uint8)
s_new = F[s]
img_new = cv.merge([h,s_new,v])
img_rgb = cv.cvtColor(img_new, cv.COLOR_HSV2RGB)
fig, axes = plt.subplots(1,3, figsize=(19,4))
axes[0].plot(i, F);axes[0].set_title("Vibrance Transformation Curve")
axes[0].set_xlabel("Input Saturation");axes[0].set_ylabel("Output Saturation")
axes[0].grid(True) # Adds a grid for better readability
axes[1].imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB)); axes[1].xaxis.set_ticks_position('top') ;
axes[1].set_title("Original Image")
axes[2].imshow(img_rgb); axes[2].xaxis.set_ticks_position('top')
axes[2].set_title(f"Transformed Image - a: {a}")
plt.tight_layout()
plt.show()
```



To achieve a visually pleasing output that enhances the mid-tone colors of the image without oversaturating the bright reds, I adjusted the parameter to $a = 0.4$

In []: